SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 20%

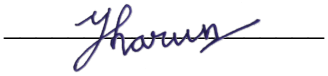
Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**

Question Count: 5

Code of Conduct

I Tharunkumar.S / 192511416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

Requirements and design choice

QUESTION 1

A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

1. Scenario interpretation

The demanding path is video delivery, not merely user login. Millions of users simultaneously retrieving multi-second segments can generate terabits per second of aggregate traffic. Separate the media data plane (CDN and object storage) from the application control plane (catalog, authorization and session APIs).

The system must prevent playback stalls, adapt to varied network speeds, absorb sudden audience surges and avoid a single congested origin. The design must scale in several geographic regions while keeping copyrighted content protected and expenditure proportionate to real demand.

2. Functional and non-functional requirements

Requirement	Recommended engineering choice	How it is verified
Concurrent playback	CDN edge distribution plus regional origin shields	Edge offload and stable startup time
Minimal buffering	HLS/DASH adaptive bitrate renditions, segment caching	Rebuffer ratio and time-to-first-frame
Traffic bursts	Global routing, autoscaling stateless APIs and origins	p95 API latency at peak load
Content protection	TLS, signed URLs, DRM for licensed assets	Reject expired and forged tokens
Cost governance	Tiered storage, efficient encoding, cache-hit tuning	Egress and compute cost per watch-hour

3. Proposed approach and justification

Choose object storage for durable media assets, packaging and encoding workers for bitrate ladders, geographically distributed CDN points of presence, origin shielding, and stateless orchestration for account and entitlement APIs. Do not serve every viewer directly from virtual machines.

> A CDN reduces the number of origin requests, but a cache miss or synchronized live event can still overload the origin. Use cache key normalization, controlled TTLs, pre-warming for planned premieres, multi-origin failover and independent telemetry for edge and origin behavior.

Streaming delivery architecture

4. Proposed system architecture diagram

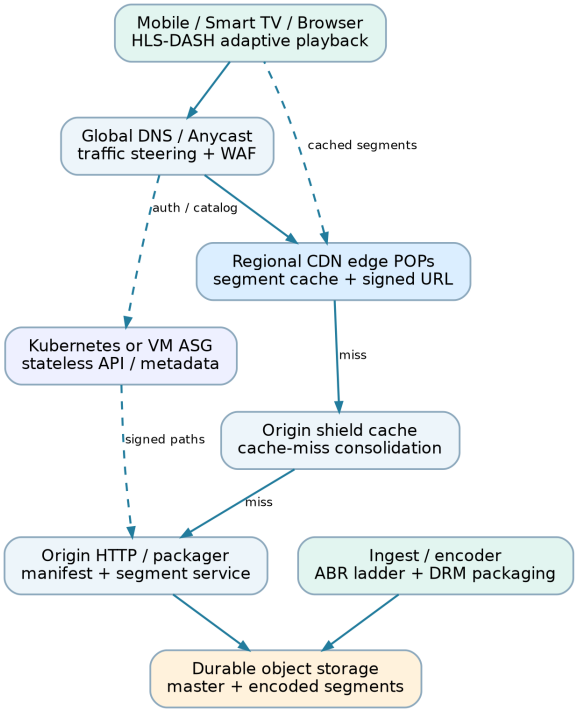


Figure 1.1: Reference architecture for the scenario. Solid arrows represent principal data or request paths; dashed arrows represent control, replication, or monitoring relationships.

5. Responsibilities of the architecture components

Component	Responsibility and technical detail
Viewer and ABR player	Requests manifests and short chunks; selects bitrate based on throughput and buffer level.
Global DNS and WAF	Chooses healthy delivery region; filters abusive requests; routes API traffic.
CDN edge / origin shield	Caches popular segments near users and consolidates upstream cache misses.
Origin / packager	Serves uncached manifests and chunks from replicated object storage.
Encoder and media store	Creates bitrate ladder; encrypts when needed; stores immutable segment objects.
API autoscaling tier	Runs stateless catalog, authentication and entitlement services independently.

6. End-to-end data and control flow

A viewer resolves a nearby edge, authenticates against a stateless API, receives a bounded signed playback URL, and requests a manifest. The player then downloads segments primarily from the CDN. A missed object is fetched through the origin shield from object storage; new renditions are produced out of band.

Load balancing and scaling workflow

7. Operational sequence / control workflow

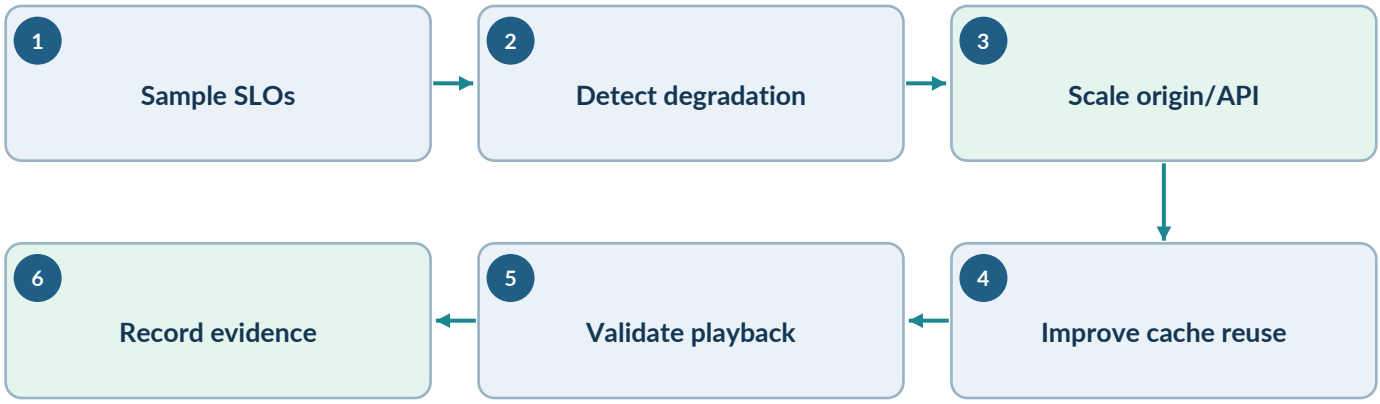


Figure 1.2: Proposed operating sequence. The steps describe a repeatable process; production implementations also require retry paths, error reporting and approved rollback decisions.

8. Procedure and control logic

Use autoscaling based on request rate, queue depth, CPU and p95 response time with cooldowns and maximum replica limits. CDN egress does not scale by adding API replicas, so separate playback, caching, and control metrics.

Security and availability boundaries: put private origins behind CDN-only access, separate transcoding queues from viewer-facing APIs, use regional replication for media assets, and health-check the cache/origin chain independently. CDN capacity is a service assumption that still requires contract limits and event rehearsal.

9. Key observability and performance indicators

Indicator / control	What to collect and why
Time to first frame	Trace manifest retrieval, authorization and first segment; test mobile and low-bandwidth clients.
Buffering ratio	Playback stall duration divided by session duration; examine p95 during flash crowds.
CDN hit ratio	Edge/object and byte-hit rate; investigate cold segments and ineffective cache keys.
Origin saturation	Monitor upstream egress, 5xx, queue depth and origin shield connection count.
Autoscale response	Load ramp, cooldown correctness and replica placement across failure zones.

10. Engineering calculation or planning example

Illustrative sizing (not a measured production result): 1,000,000 concurrent viewers at an average delivered rate of 4 Mb/s imply about 4 Tb/s of aggregate delivery. If 95% of transferred bytes are served by edge caches, the remaining 5% represents about 200 Gb/s reaching upstream cache layers before further shielding and packaging savings. The illustration shows why CDN offload is essential.

Reliability, cost and verification

11. Main risks, mitigations and evidence

Failure or risk	Proposed mitigation	Evidence / test
Live-event cache stampede	Pre-warm common chunks, use request collapsing and origin shields	Load test cold and warmed releases
Origin or region outage	Multi-origin failover and cross-region object replication	Fail origin in a controlled drill
Unbounded egress spend	Budgets, adaptive bitrate policy, cache efficiency analysis	Cost per delivered GB/watch-hour
Playback piracy and abuse	DRM where required, signed URLs, WAF, token TTLs	Try replay and expired URL cases
Transcode backlog	Queue-based workers, retry/dead-letter handling	Measure encode lag at input peaks

12. Validation and acceptance test plan

Test scenario	Method	Expected acceptance criterion
Capacity rehearsal	Synthetic viewers ramp to expected peak; exercise mixed bitrates	No uncontrolled p95 stalls; document actual limit
Regional failover	Disable primary origin/API region and observe routing	Service recovers within chosen continuity objective
Security review	Test token expiry, unauthorized origin access and DDoS controls	Unauthorized playback/origin access denied
Cost verification	Compare per-region traffic, storage classes, requests and encoding	Approved spend envelope with alerts

13. Architectural trade-offs and decision rationale

A CDN reduces the number of origin requests, but a cache miss or synchronized live event can still overload the origin. Use cache key normalization, controlled TTLs, pre-warming for planned premieres, multi-origin failover and independent telemetry for edge and origin behavior.

14. Final assessment answer / conclusion

A CDN-first architecture with ABR playback, shielded origins and independent autoscaling control services best meets the massive-concurrency requirement. Capacity should be proven through playback-focused load tests, not inferred from CPU utilization alone.

Assessment note: proposed designs and illustrative thresholds require validation against the organization's real workload, SLA, risk appetite, cloud-provider capabilities and budget. No unmeasured result is represented as an observed benchmark.

Consistency and orchestration strategy

QUESTION 2

A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

1. Scenario interpretation

Container images make an application package portable, but they do not automatically make its runtime configuration, networking, identity, persistent volumes and managed services portable. A provider-neutral deployment process is required to prevent drift between cloud environments.

The objective is repeatable releases into two provider-specific clusters without inconsistent services or excessive operator intervention. The proposed boundary is a common Kubernetes deployment specification with approved environment overlays and independent workload identity in each cloud.

2. Functional and non-functional requirements

Requirement	Recommended engineering choice	How it is verified
Deployment consistency	Signed immutable images, Helm/Kustomize and GitOps	Same image digest and reviewed configuration
Service discovery	Kubernetes DNS/Services within each cluster	Service names resolve and endpoints update
Elastic capacity	HPA/KEDA for pods and cluster autoscaler for nodes	Correct scale-out without resource starvation
Cross-cloud traffic	External health-aware DNS or global routing	Requests avoid unhealthy provider
Trust and observability	OIDC federation, RBAC, secrets, central telemetry	Audit, logs and least-privilege controls

3. Proposed approach and justification

Use separate managed Kubernetes clusters in Cloud A and Cloud B with a shared GitOps repository and OCI image registry. Keep provider-specific storage, load balancer and identity adapters explicit rather than hiding differences. Prefer stateless application portability before planning data mobility.

> Multi-cloud reduces reliance on a single provider and supports regional/business continuity goals, but increases operational surface area, network egress, compliance checks and distributed-data complexity. Adopt it only when these benefits justify the ongoing cost.

Multi-cloud Kubernetes architecture

4. Proposed system architecture diagram

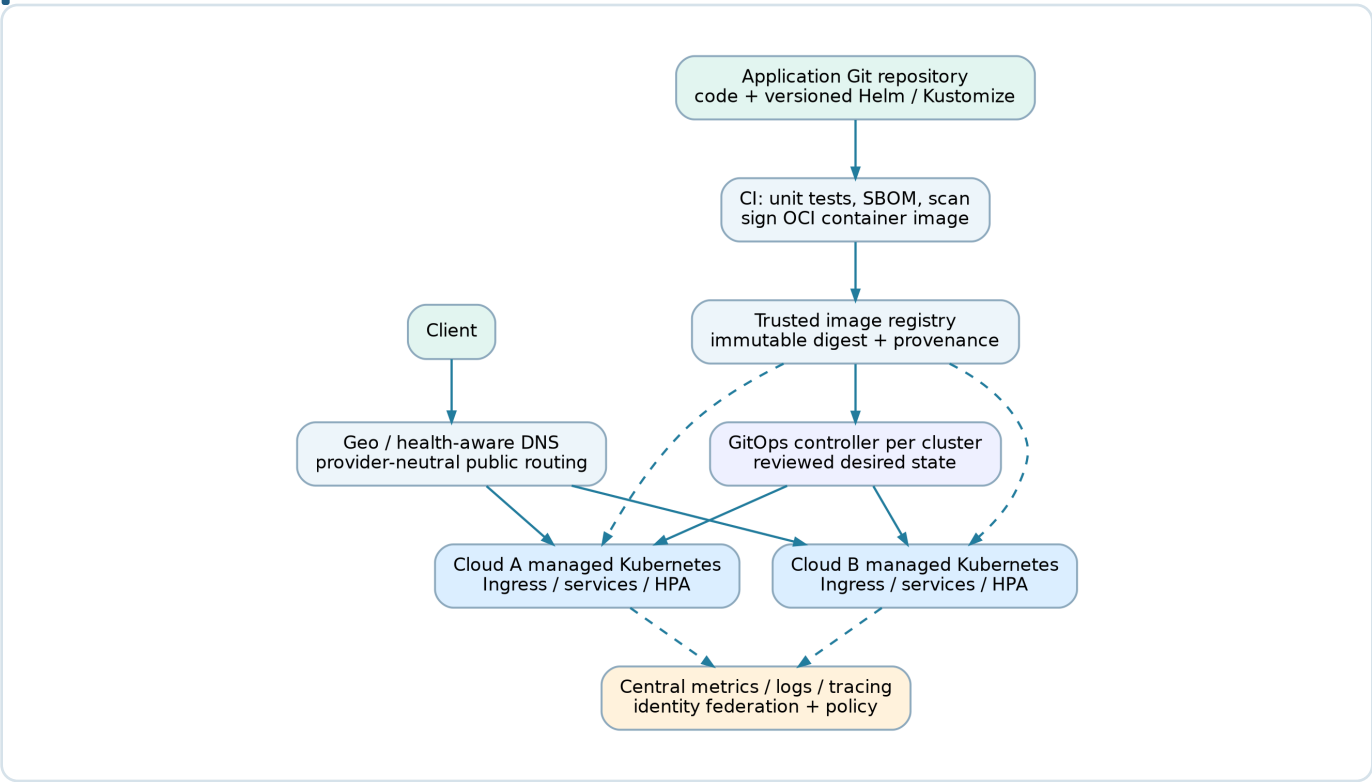


Figure 2.1: Reference architecture for the scenario. Solid arrows represent principal data or request paths; dashed arrows represent control, replication, or monitoring relationships.

5. Responsibilities of the architecture components

Component	Responsibility and technical detail
CI + OCI registry	Build/test/scan once; publish image by immutable digest and signed provenance.
GitOps controllers	Reconcile reviewed desired state for each cluster; detect configuration drift.
Managed Kubernetes A / B	Run deployments, ingress, readiness probes, network policy and HPA.
Cluster service DNS	Discover in-cluster services using stable names and Kubernetes Service objects.
Global routing layer	Select healthy cluster/provider; do not rely on internal cluster DNS across clouds.
Identity and telemetry	Use workload federation and central logs, metrics, traces, alerts and policy checks.

6. End-to-end data and control flow

A developer merges a reviewed change; CI produces a signed image and writes its digest to a release manifest. GitOps controllers in each cloud reconcile resources, using explicit overlays for ingress, storage and secret references. Readiness probes keep unhealthy pods out of service, and global traffic routing gradually shifts clients.

Deployment and service-discovery workflow

7. Operational sequence / control workflow



Figure 2.2: Proposed operating sequence. The steps describe a repeatable process; production implementations also require retry paths, error reporting and approved rollback decisions.

8. Procedure and control logic

Within each cluster, service discovery uses CoreDNS and Services. Horizontal Pod Autoscaler (HPA) can target CPU or custom metrics, while event-driven workloads use KEDA. Cluster Autoscaler adds nodes when pods cannot be scheduled. Across clusters, use health-aware external routing and explicit failover policies.

Separate cluster failure domains and network policies. Mirror registry artifacts if provider outages threaten image pulls. Avoid exporting long-lived cloud keys into pods; use short-lived cloud-native workload credentials and encrypted secrets with audited rotations.

9. Key observability and performance indicators

Indicator / control	What to collect and why
Release consistency	Compare desired vs running image digests, configuration revisions and policy results.
Internal discovery	Kill/recreate pods and check that Service routing finds healthy endpoints.
Scale-out speed	Load test HPA / node scale-up; record pending pods and cold start.
Cross-cloud continuity	Make Cloud A unhealthy; verify retries, state handling and route changes.
Security posture	Admission policy, image verification, RBAC scope and network-policy denial.

10. Engineering calculation or planning example

Illustrative capacity estimate (not a benchmark): 6 pods at a tested 100 requests/s each yield about 600 requests/s before headroom. A 1,200 requests/s peak would require at least 12 equivalent pods, plus reserve and dependency-capacity checks.

Risks, benefits and acceptance tests

11. Main risks, mitigations and evidence

Failure or risk	Proposed mitigation	Evidence / test
Cluster configuration drift	GitOps reconciliation, PR review and audit history	Deliberately change a live object; detect drift
Data gravity and egress	Place write-heavy datasets deliberately; avoid chatty cross-cloud calls	Measure cross-provider traffic cost
Identity inconsistency	Issuer/audience mapping, scoped service accounts, token rotation	Reject wrong-audience credentials
Supply-chain compromise	Signed image verification, SBOM and admission checks	Block untrusted digest
Provider-specific APIs	Isolate adapters and test each provider overlay	Deploy both clusters from clean state

12. Validation and acceptance test plan

Test scenario	Method	Expected acceptance criterion
End-to-end deploy	Promote a new image into both clusters using GitOps	Identical application release with valid overlays
Horizontal scale	Saturate a service and confirm HPA and node scaling	Scale preserves latency and no unschedulable backlog
Outage simulation	Disable ingress or one provider cluster	Healthy provider serves supported traffic
Security & rollback	Inject bad image/policy and regress canary SLO	Admission denies or rollout reverts correctly

13. Architectural trade-offs and decision rationale

Multi-cloud reduces reliance on a single provider and supports regional/business continuity goals, but increases operational surface area, network egress, compliance checks and distributed-data complexity. Adopt it only when these benefits justify the ongoing cost.

14. Final assessment answer / conclusion

Managed Kubernetes with GitOps, immutable container images, provider-specific adapters and health-aware routing creates a consistent multi-cloud operating model. Portability claims must include identity, data and operations—not containers alone.

Assessment note: proposed designs and illustrative thresholds require validation against the organization's real workload, SLA, risk appetite, cloud-provider capabilities and budget. No unmeasured result is represented as an observed benchmark.

Data-loss requirements and recovery objectives

QUESTION 3

An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning. Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

1. Scenario interpretation

Accidental deletion is a logical data error that can replicate rapidly to read replicas or a second region. Replication improves availability but does not replace backups. A resilient e-learning system needs independently protected historical copies, tested restore procedures and business continuity plans.

The protected datasets include enrollment records, courses, grades, assignment files, authentication-related configuration and audit trails. Prioritize recovery by business impact: grading and submission records may require tighter objectives than public course thumbnails.

2. Functional and non-functional requirements

Requirement	Recommended engineering choice	How it is verified
Recover deleted content	Object versioning and time-bound retention	Restore a previous assignment/course version
Transactional recovery	DB point-in-time recovery using log archiving	Restore to an exact approved timestamp
Regional resilience	Cross-region standby and independent backup storage	Recover after primary-region loss
Malicious/accidental deletion protection	Immutable, least-privilege backup vault	Production admin cannot erase recovery copies
Continuity objectives	Document per-service RTO and RPO; drill regularly	Measured restore time and recovery gap

3. Proposed approach and justification

Enable object versioning, DB snapshots with transaction-log archiving, immutable cross-account/offsite backup copies and tested infrastructure-as-code recreation. Use a warm regional standby for critical services where lower RTO is justified. Document the key backup encryption ownership and restore authorization workflow.

> Synchronous replication may reduce data loss but adds latency and coupling; asynchronous replication can lose recent writes. Restore from a protected version may require data consistency checks across database rows and file objects. Define a coherent recovery point instead of mixing unmatched backups.

Backup, replication and immutable protection

4. Proposed system architecture diagram

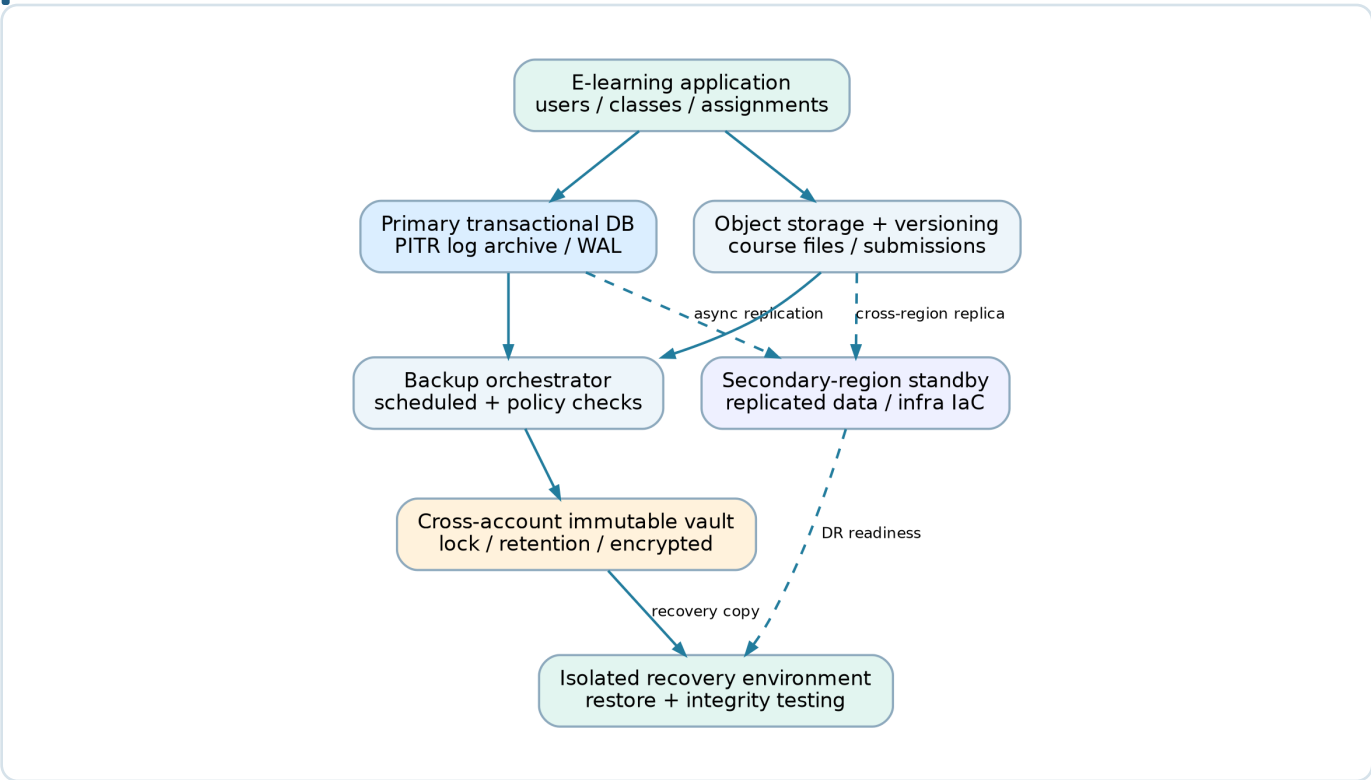


Figure 3.1: Reference architecture for the scenario. Solid arrows represent principal data or request paths; dashed arrows represent control, replication, or monitoring relationships.

5. Responsibilities of the architecture components

Component	Responsibility and technical detail
Transactional database	PITR-enabled database and continuous transaction-log backup.
Versioned object store	Retains prior course/assignment file versions and deletion markers.
Backup coordinator	Schedules snapshots; inventories backup sets; verifies job success.
Immutable recovery vault	Cross-account encrypted backup with restricted delete controls.
Regional standby	Receives replicated workloads and data; remains ready for failover.
Recovery environment	Isolated restore and test network used before cutover.

6. End-to-end data and control flow

A normal write is committed to the primary database and course file store; change streams and snapshots are stored under separate backup policies. A second-region copy protects against site failure, while an immutable backup vault protects against accidental or malicious change propagation.

Restore procedure and RTO/RPO workflow

7. Operational sequence / control workflow



Figure 3.2: Proposed operating sequence. The steps describe a repeatable process; production implementations also require retry paths, error reporting and approved rollback decisions.

8. Procedure and control logic

For isolated accidental deletion, restore only affected records when referential integrity can be guaranteed. For widespread corruption, restore a consistent database and object-state pair into staging, run checks, then redirect application traffic. Record each decision to protect auditability.

A snapshot alone is insufficient if it is never restored. Routinely check backup catalog completeness, integrity, encryption-key availability, object version lifetimes and the permissions needed to restore into a clean account.

9. Key observability and performance indicators

Indicator / control	What to collect and why
RPO	Maximum acceptable gap between last protected commit and incident time.
RTO	Maximum allowed elapsed time from incident start to useful restored service.
Data consistency	Grades, course IDs and submission objects reconcile after restoration.
Backup independence	Primary-role credentials cannot modify immutable recovery objects.
Restore repeatability	Run scheduled exercises from backups into a disposable isolated environment.

10. Engineering calculation or planning example

Illustrative business targets (proposed, not present in the question): set an RPO of 15 minutes and RTO of 60 minutes for critical assessment data. If a failure starts at 14:00, an eligible restore point at 13:50 gives a 10-minute recovery gap (within RPO); the platform must be usable again by 15:00 to meet the one-hour RTO. Actual targets require business approval.

Business-continuity tests and governance

11. Main risks, mitigations and evidence

Failure or risk	Proposed mitigation	Evidence / test
Replicated deletion	Versioned and immutable backup history separated from replication	Delete primary record; recover prior version
Corrupted restore copy	Hash/consistency validation plus multiple recovery points	Perform restore integrity drill
Lost encryption keys	Protected recovery keys, access segregation and key escrow plan	Restore using disaster credentials
Restore too slow	Warm standby for critical tiers and rehearsal of runbook	Time full recovery against RTO
Inconsistent files/database	Application-consistent snapshots and reconciliation	Cross-check grade and submission links

12. Validation and acceptance test plan

Test scenario	Method	Expected acceptance criterion
Single file deletion	Restore past version of student submission	Correct original content and metadata returned
Database point-in-time	Restore to time just before accidental SQL deletion	Recovered records and referential consistency
Region outage	Bring up standby from IaC and current protected data	Service within approved RTO/RPO
Immutability control	Attempt unauthorized backup deletion	Deletion denied and recorded in audit

13. Architectural trade-offs and decision rationale

Synchronous replication may reduce data loss but adds latency and coupling; asynchronous replication can lose recent writes. Restore from a protected version may require data consistency checks across database rows and file objects. Define a coherent recovery point instead of mixing unmatched backups.

14. Final assessment answer / conclusion

The defensible solution combines versioning, point-in-time database recovery, immutable offsite copies, controlled replication and repeated restore drills. RPO and RTO must be measurable commitments supported by evidence, not merely stated numbers.

Assessment note: proposed designs and illustrative thresholds require validation against the organization's real workload, SLA, risk appetite, cloud-provider capabilities and budget. No unmeasured result is represented as an observed benchmark.

Data classification and compliance boundaries

QUESTION 4

A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

1. Scenario interpretation

A hybrid cloud places selected applications and data under government-controlled on-premises infrastructure while using cloud resources for approved services. The architecture must separate data by sensitivity and ensure that every transfer across domains is explicitly authorized, encrypted and recorded.

On-premises retention alone does not guarantee compliance. Required controls may include sovereign/residency policies, auditability, departmental segregation, key ownership, procurement restrictions and incident response. Each dataset must be classified before determining its allowed location and movement.

2. Functional and non-functional requirements

Requirement	Recommended engineering choice	How it is verified
Sensitive data locality	Protected on-prem zone for regulated records	Data residency and export controls
Secure communications	Redundant private links / IPsec VPN plus application TLS	Link failover and interception resistance
Identity consistency	Central SSO federation, MFA, RBAC/ABAC, workload identity	Cross-environment least privilege
Cryptographic control	TLS 1.3, KMS/HSM keys, encrypted volumes/backups	Key rotation and recovery audits
Visibility and compliance	Policy as code, SIEM, configuration evidence	Access trails and misconfiguration alerts

3. Proposed approach and justification

Classify data into sensitive and permitted-cloud tiers. Place confidential record systems and authoritative databases on premises. Deploy publicly scalable portals, approved analytics or stateless APIs in cloud private networks connected by redundant private transport. Prohibit unrestricted database copies between zones.

> Hybrid architecture provides control over sensitive assets and flexible capacity for approved functions, but doubles the number of network, identity and management planes. The most difficult issue is avoiding policy inconsistency when workloads span two independently operated environments.

Hybrid trust-zone and network architecture

4. Proposed system architecture diagram

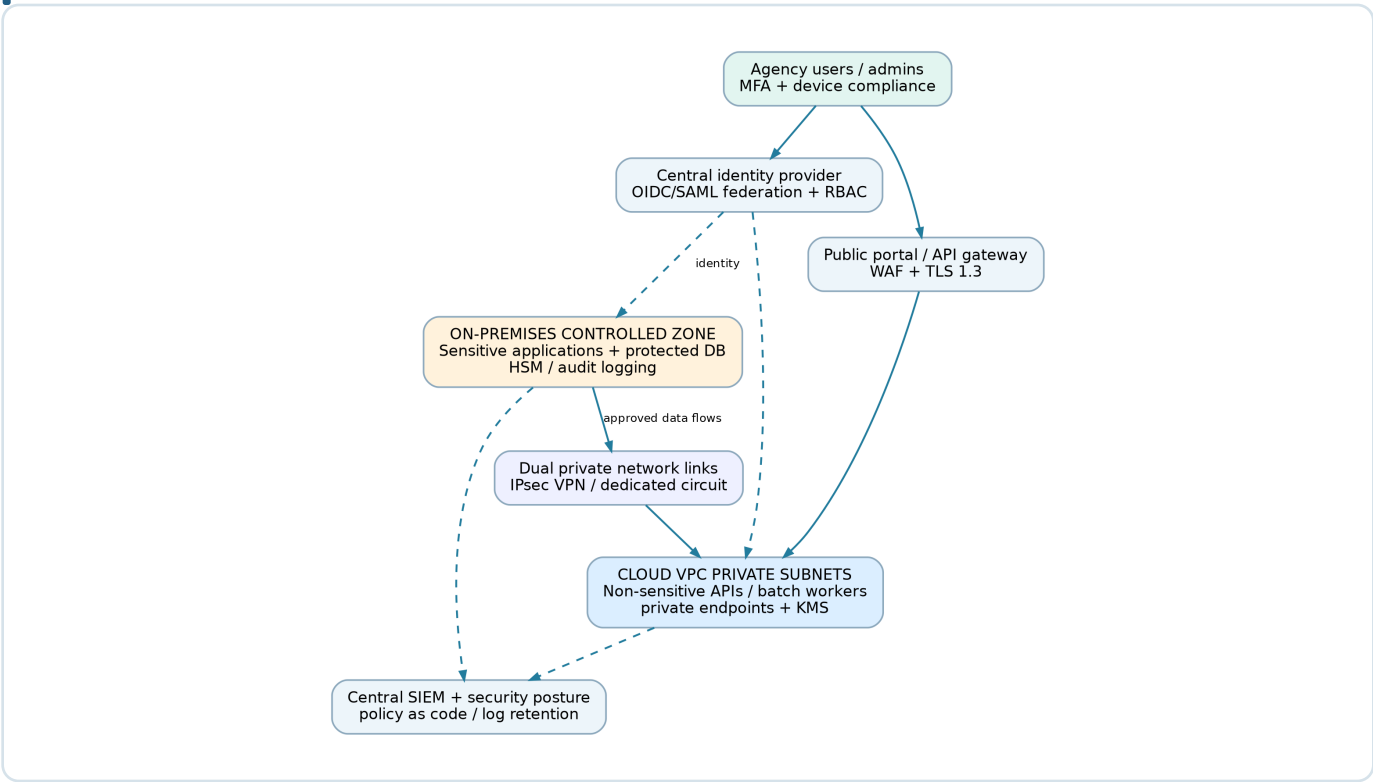


Figure 4.1: Reference architecture for the scenario. Solid arrows represent principal data or request paths; dashed arrows represent control, replication, or monitoring relationships.

5. Responsibilities of the architecture components

Component	Responsibility and technical detail
On-prem protected zone	Confidential applications and data under local policy and physical control.
Cloud VPC/subnets	Approved stateless applications; private endpoints; restricted egress.
Private connectivity	Redundant dedicated link or IPsec VPN with route failover testing.
Authoritative IdP	User federation, phishing-resistant MFA and lifecycle provisioning.
KMS/HSM	Separate keys and access policies for regulated and public data classes.
Central SOC / SIEM	Correlates identity, cloud, on-prem, network and key-management logs.

6. End-to-end data and control flow

A user signs in through the authoritative IdP and receives a short-lived audience-bound identity. A cloud API validates access rights before calling a protected on-prem service over an approved encrypted private link. The on-prem policy decision point limits response fields and logs the transfer.

Identity, encryption and access workflow

7. Operational sequence / control workflow

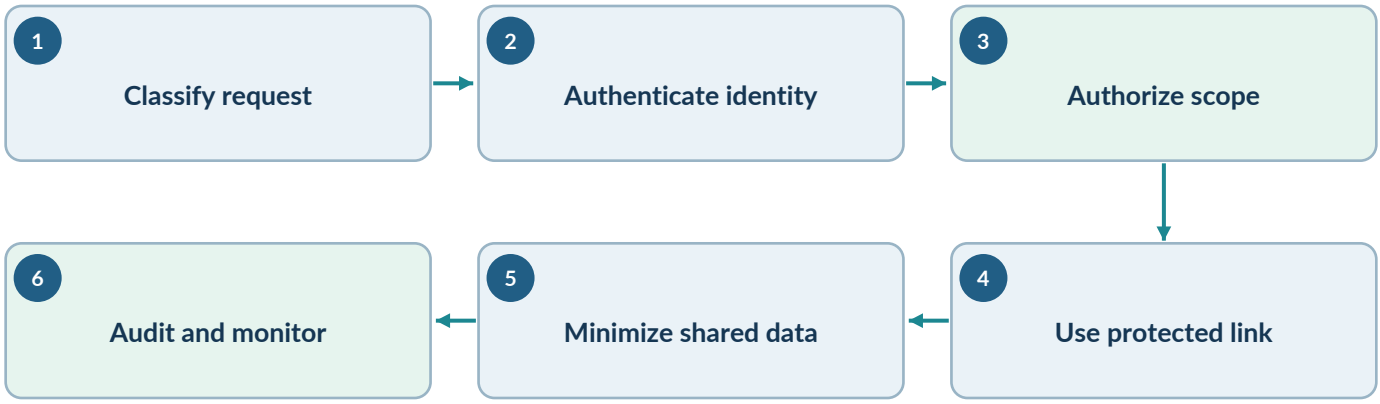


Figure 4.2: Proposed operating sequence. The steps describe a repeatable process; production implementations also require retry paths, error reporting and approved rollback decisions.

8. Procedure and control logic

Identity checks should separate authentication (who or what made the request) from authorization (which action on which resource is permitted). Cloud workloads should use federated short-lived tokens rather than exported static service-account credentials.

Do not assume a private circuit is encrypted end-to-end; still use strong TLS. Network segmentation, conditional access, workload identity and device posture checks remain essential. Keep high-sensitivity data from appearing in developer logs and cross-region caches.

9. Key observability and performance indicators

Indicator / control	What to collect and why
Data-residency conformance	Scan object locations, backups and logs for prohibited sensitive copies.
Cross-boundary authorization	Try access using unauthorized user, role, app and workload identities.
Transport security	Validate encryption, certificates, private routing and key rotation.
Hybrid latency and availability	Measure p95 intersite API response and private-link failover.
Incident reconstruction	Correlate transaction, IAM, KMS and firewall audit events end-to-end.

10. Engineering calculation or planning example

Illustrative connectivity resilience: two independently routed private connections offer better availability only when they do not share a single physical carrier, gateway or on-prem edge device. A pair of connections in the same conduit can still fail together. Verify actual failure-domain independence with provider documentation and cutover exercises.

Operational challenges and security assurance

11. Main risks, mitigations and evidence

Failure or risk	Proposed mitigation	Evidence / test
Credential and role drift	Central federation, scoped roles and JIT provisioning	Revoke user and verify access loss
Sensitive data leakage	Classification enforcement, DLP, minimum-necessary API responses	Attempt forbidden exports
Connectivity outage	Dual routing paths and local fail-safe behavior	Disconnect one transport path
Inconsistent patching	Unified posture inventory and policy-as-code controls	Simulate missing security patch
Key/secret compromise	Managed keys, rotation, short-lived tokens, HSM where required	Test revocation and rotation

12. Validation and acceptance test plan

Test scenario	Method	Expected acceptance criterion
Least privilege	Access protected record from unauthorized cloud role	Access denied and logged
Encryption validation	Trace cross-zone calls and inspect key policy	TLS + at-rest protection enforced
Link failover	Interrupt one VPN or private circuit	Approved traffic uses surviving route
Audit drill	Correlate a specific user request in both environments	Trace complete with consistent timestamps

13. Architectural trade-offs and decision rationale

Hybrid architecture provides control over sensitive assets and flexible capacity for approved functions, but doubles the number of network, identity and management planes. The most difficult issue is avoiding policy inconsistency when workloads span two independently operated environments.

14. Final assessment answer / conclusion

A secure hybrid solution depends on explicit data classification, protected on-prem databases, encrypted private connectivity and common federated identity with local authorization. Strong operations require unified monitoring, tested failure paths and compliance evidence.

Assessment note: proposed designs and illustrative thresholds require validation against the organization's real workload, SLA, risk appetite, cloud-provider capabilities and budget. No unmeasured result is represented as an observed benchmark.

Failure objectives and region topology

QUESTION 5

A SaaS provider needs to ensure high availability and fault tolerance for its application. The system must continue operating even if one region goes down. Propose a multi-region deployment strategy in the cloud. Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

1. Scenario interpretation

Availability across zones protects against local data-center failures; surviving a complete region outage requires a second operating region with network, compute and recoverable data. The proposed design distinguishes the stateless application plane from the stateful database write plane.

The SaaS provider must keep user-facing functions available when a region fails while preventing duplicate transactions, conflicting database writers and stale traffic routing. Cross-region redundancy alone is not enough: failover must be deterministic and tested.

2. Functional and non-functional requirements

Requirement	Recommended engineering choice	How it is verified
Regional resilience	Two independent regions each spanning multiple availability zones	App remains reachable after full-region loss
Traffic failover	Global health-aware routing and synthetic probes	Client traffic shifts to healthy endpoints
Data continuity	Replicated database, independent backups, replication-lag monitoring	Declared RPO and verified data integrity
Safe write semantics	Writer fencing, idempotency keys and event outbox	No split-brain or duplicate billing
Operational detection	SLIs/SLOs, centralized logs, paging and drills	Actionable alerts before broad impact

3. Proposed approach and justification

Use active-passive write topology for a typical transaction-heavy SaaS application: Region A runs the authoritative database writer, and Region B maintains a warm, ready standby with read replica. Deploy stateless application replicas in both regions and use health-aware global routing. Promote the standby only after old-writer fencing.

> Active-active writes can provide better distribution but need difficult conflict resolution and consistent business semantics. Active-passive is simpler for financial, order and account state, though regional failover has measurable lag and recovery time. No topology automatically provides zero RPO with asynchronous replication.

Primary/standby and replication architecture

4. Proposed system architecture diagram

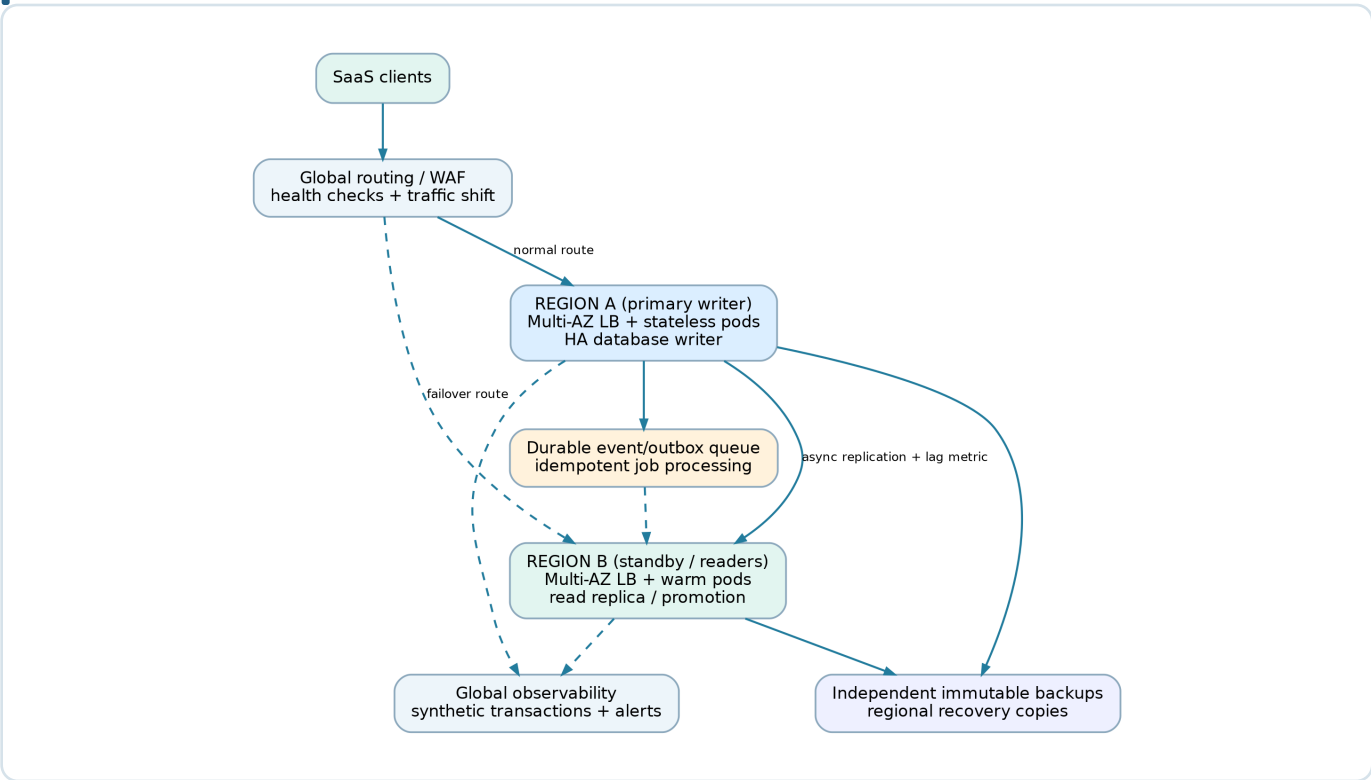


Figure 5.1: Reference architecture for the scenario. Solid arrows represent principal data or request paths; dashed arrows represent control, replication, or monitoring relationships.

5. Responsibilities of the architecture components

Component	Responsibility and technical detail
Global traffic router	Routes client requests based on health and geography; honors TTL limits.
Multi-AZ application tier	Runs healthy stateless pods/VMs in both regions with capacity reserve.
Primary writer and standby	Single-write authority with replica lag telemetry and promotion procedure.
Outbox / event processor	Guarantees idempotent retries and eventual background job completion.
Independent recovery copies	Versioned immutable backups in separate failure domains.
Operations control plane	Synthetic checks, telemetry, incident automation and audited failover.

6. End-to-end data and control flow

Normally a global router directs writes to Region A, while Region B remains prepared to serve requests. Database change records replicate to B; background events are processed with unique IDs. If the regional health quorum declares A unavailable, the runbook fences its writer, promotes B and shifts global traffic.

Automatic failover with split-brain prevention

7. Operational sequence / control workflow

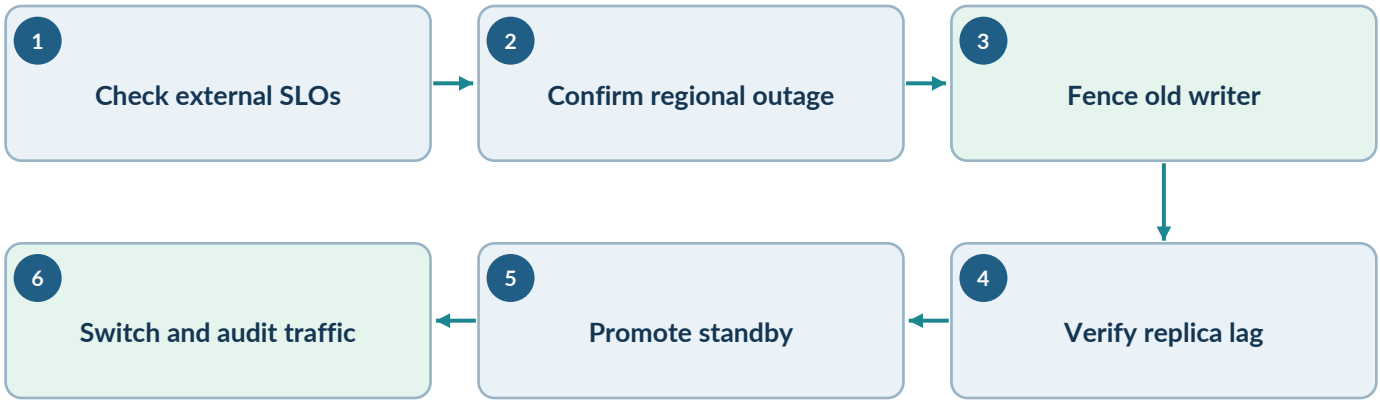


Figure 5.2: Proposed operating sequence. The steps describe a repeatable process; production implementations also require retry paths, error reporting and approved rollback decisions.

8. Procedure and control logic

Implement database promotion with explicit write fencing and an ownership lease/quorum to prevent two primaries. Application deployments must not assume a fixed region name for the writer. The DNS or global load balancer must redirect only after healthy read/write verification.

A routing probe must distinguish application failure from network visibility problems. Use multi-signal detection and a controlled failover threshold to avoid flapping. For non-idempotent requests, retry only when a transaction identifier permits safe deduplication.

9. Key observability and performance indicators

Indicator / control	What to collect and why
Availability SLO	Measure successful user operations, not just VM or load balancer uptime.
Replication lag	Track bytes/time behind primary and alarm before RPO breach.
Failover interval	Time from incident detection to verified customer operation in secondary region.
State integrity	Compare committed records and idempotent retries after failover.
Monitoring quality	Alert precision, detection time, escalation and drill completion evidence.

10. Engineering calculation or planning example

Illustrative objectives (subject to the SaaS product contract): RTO = 10 minutes and RPO = 5 minutes. If the last applied replicated commit in Region B trails Region A by 90 seconds at cutover, the estimated data gap is below the five-minute RPO, but lost or in-flight transactions still require reconciliation. Verify lag from actual commit positions, not DNS timing.

Monitoring, drills and decision justification

11. Main risks, mitigations and evidence

Failure or risk	Proposed mitigation	Evidence / test
Split-brain writers	Fence previous primary; lease/quorum and promotion guards	Simulate asymmetric network partitions
Stale replica writes	Lag alerts, RPO threshold and client reconciliation	Fail with controlled replication delay
DNS / client stickiness	Health-aware low-TTL routing and retry policy	Measure real client switch time
Duplicate requests	Idempotency keys and transactional outbox	Replay payment/order requests
Alert fatigue or blind spots	User-journey probes, error budgets and graded escalation	Inject synthetic failure signatures

12. Validation and acceptance test plan

Test scenario	Method	Expected acceptance criterion
Full region outage	Disable Region A app and database writer under supervision	Region B serves writes within agreed RTO
Partition safety	Separate regions without fully failing old writer	Never permit simultaneous write primaries
Data correctness	Compare transaction counts and replay pending events	No unreconciled duplicates or missing durable events
Observability	Trigger elevated errors, lag and capacity pressure	On-call receives actionable, timestamped alerts

13. Architectural trade-offs and decision rationale

Active-active writes can provide better distribution but need difficult conflict resolution and consistent business semantics. Active-passive is simpler for financial, order and account state, though regional failover has measurable lag and recovery time. No topology automatically provides zero RPO with asynchronous replication.

14. Final assessment answer / conclusion

The proposed warm-standby multi-region design uses independent application capacity, measured replication lag, controlled writer promotion and verified global routing. Safe failover requires regular drills, not just a second region on an architecture diagram.

Assessment note: proposed designs and illustrative thresholds require validation against the organization's real workload, SLA, risk appetite, cloud-provider capabilities and budget. No unmeasured result is represented as an observed benchmark.

SCENARIO-BASED RUBRIC

Original evaluation table reproduced directly from the uploaded assessment PDF.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

Rubric weighting (as printed):

Understanding 20% | Risks / Issues 20% | Concepts 25% | Decision 20% | Clarity 15%